

Last modified on

Sports Data Soccer API - In-Venue Aggregated Fitness Feed (MA45)

Get ‘Aggregated Fitness’ data for a Soccer match collected in venue.

Overview

The Soccer **In-Venue Aggregated Fitness** feed will provide aggregated fitness data collected in venue during the course of a game. The feed will contain all traditional data such as match information, competition information, details of the team that are part of this match, goals, substitute, period, score related details and venue informations, clubbed with new information like Aggregated Fitness Data.

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you do not exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 13	TIER 8+9+10+11	TIER 6+7	TIER 5	TIER 1+2+3
Updated on every stop in play during the game, including for the following types of events: goal, shot at goal, assist, free kick, offside given, cards, start of the half, end of the half, corner, substitution, lineup Note: There will always be a gap of at least 30 seconds between files unless a goal is scored	See Tier 13	Post-match	n/a	No more frequently than one URL poll every 30 seconds for range: <KO - 1h; KO + 4h> Up to 1-hour interval for range: <KO + 4h; KO + 8h> Up to 24-hour interval 8h+ of match ending, lasting a maximum of 7 days	n/a	n/a	n/a	n/a

In-Venue Aggregated Fitness (MA45) Guide

USAGE

GET in venue collected **Aggregated Fitness Data** information for a fixture by passing the fixture/match UUID as a path parameter (the a

GET in venue collected **Aggregated Fitness Data** information for a fixture by passing a fixture UUID to the fx parameter.)

If your request URL is as follows (does not contain the fixture UUID query criteria), the feed returns an error:
/soccerdata/invenueaggregatedfitness/{outletAuthKey}

How can I get fixture and other entity UUIDs?

Use the **Tournament Calendar (OT2)** or **Fixtures and Results (MA1)** feeds to get fixture metadata, including UUIDs for each fixture and its linked contestants, competition, tournament calendar and other entities.

Query Parameters And Example Request Parameters

This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/invenueaggregatedfitness` feed by a fixture (match) UUID.

Legacy IDs: You can use the `fx` query parameter to filter by legacy IDs (only one ID is supported - you cannot specify a comma-separated list of IDs).

How to use query parameters and make successful requests:

In our guides, we use HTTPS syntax for our example `GET` requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You **MUST** format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the **main guide** and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format). If required, it should also include an entity UUID (such as a fixture/match UUID).
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{}` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you **MUST** use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
fixture/match UUID + type	GET in venue collected Aggregated Fitness Data information for fixture UUID by passing the fixture/match UUID as a path parameter (the asset/resource) along with type parameter in the URL. Note: In this feed, you must use the type parameter (required). For now, type can have one of following 2 values – fifa historical . <pre>GET https://api.performfeeds.com/soccerdata/invenueaggregatedfitness/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&type={fifa historical}</pre>
fx + type	GET in venue collected Aggregated Fitness Data information for a fixture UUID by passing a fixture UUID to the fx parameter along with type parameter. <pre>GET https://api.performfeeds.com/soccerdata/invenueaggregatedfitness/{outletAuthKey}?fx={fixtureUuid}&_rt={mode}&_fmt={dataFormat}&type={fifa historical}</pre>

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

FIFA

- [XML](#)
- [JSON](#)

HISTORICAL

- [XML](#)
- [JSON](#)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▶<inVenueAggregatedFitness>

▶<matchInfo>

▶<description>

▶<sport>

▶<ruleset>

▶<competition>

▶<country>

▶<tournamentCalendar>

▶<stage>

▶<series>

▶<contestants> (XML only)

▶<contestant>

▶<country>

▶<venue>

▶<liveData>

▶<matchDetails>

▶<periods> (XML only)

▶<period>

▶<suspensions>

▶<suspension>

▶<scores>

▶<ht>

▶<ft>

▶<et>

▶<pen>

▶<total>

▶<totalUnconfirmed>

▶<aggregate>

▶<goal>

▶<card>

▶<substitute>

▶<fitnessStats>

▶<statType>

▶<lineUp>

▶<player>

▶<fitnessDataAggregatesOverall>

▶<stat>

▶<fitnessDataAggregatesInPossession>

▶<stat>

▶<fitnessDataAggregatesOutOfPossession>

▶<stat>

▶<teamOfficial>

▶<teamStats>

▶<fitnessDataAggregatesOverall>

▶<stat>

▶<fitnessDataAggregatesInPossession>

▶<stat>

▶<fitnessDataAggregatesOutOfPossession>

▶<stat>

▶<matchDetailsExtra>

▶<attendance>

▶<matchOfficials> (XML only)

▶<matchOfficial>

How Are VAR Events Reported?

when a VAR (Video Assistant Referee) review takes place our analysts will collect the type of event being reviewed, the team and player involved, and the decision of the review (outcome) which also describes what happens next.

Please [click here](#) for more details on VAR outcomes.

Statistic Types

Click on the buttons below to see the complete lists of player and contestant (team) statistics.

▶**Player Stats**

▶**Team Stats**